ЧАСТЬ 2. МАССИВЫ, СТРОКИ, МЕТОДЫ • ЗАНЯТИЕ 2 ИЗ 2

2.1. Массивы и список List

К концу занятия ты сможешь:

- брать элементы с конца массива и вырезать из него кусок;
- сортировать, переворачивать массив и искать в нём;
- хранить данные, количество которых заранее неизвестно, — в списке List.

Разминка

Вопрос 1

Что выведет программа?

Program.cs

```
int[] marks = { 5, 3, 4, 2 };
Console.WriteLine(marks[marks.Length - 1]);
```

Вопрос 2

Почему поиск максимума нельзя начинать с max = 0?

Вопрос 3

Когда для перебора нужен for, а когда хватит foreach?

ДВЕ ЗАДАЧИ, КОТОРЫЕ ПОКА РЕШАЮТСЯ НЕУДОБНО

Таблица рекордов и список гостей

Таблица рекордов

Очки 8 игроков записаны вперемешку. Нужно расставить их по порядку и показать тройку лучших.

Сейчас для этого нужен свой алгоритм сортировки и ещё один цикл для тройки.

Список гостей

Ты зовёшь друзей на день рождения. Кто-то соглашается, кто-то потом передумывает.

Массив на 5 мест не вырастет, когда придёт шестой гость.

Сегодня обе задачи решаются в несколько строк.

ПЛАН

План занятия

УРОК 1 • 40 МИНУТ

- Разминка и план
- Блок 1. Индексы с конца ^ и диапазоны ..
- Методы Array: Sort, Reverse, IndexOf
- Живой кодинг «Забег на 60 метров»
- Вопросы и практика «Таблица рекордов»

УРОК 2 • 40 МИНУТ

- Блок 2. Список List: Add, Remove, Count
- Правило «массив или List»
- Практика «Список покупок»
- Вопросы по всей теме
- Мини-проект «Бинго» и итоги

БЛОК 1 ИЗ 2 • УРОК 1

Берём с конца, режем и сортируем

В этом блоке ты научишься:

- брать элементы с конца через ^;
- вырезать кусок массива через диапазон ..;
- сортировать, переворачивать массив и искать в нём готовыми методами Array.

Индекс с конца: ^

^ — это «считать с конца». ^1 — последний элемент, ^2 — предпоследний. Как в очереди: «я второй с конца».

Было

```
Console.WriteLine(songs[songs.Length - 1]);
```

Стало

Program.cs

```
string[] songs = { "Believer", "Numb", "Thunder", "Faded" };
Console.WriteLine(songs[^1]); // последняя
Console.WriteLine(songs[^2]); // предпоследняя
```

Вывод в консоли

```
Faded
Thunder
```

индекс	с конца	элемент
0	^4	Believer
1	^3	Numb
2	^2	Thunder
3	^1	Faded

songs[^1] — то же, что songs[songs.Length - 1], только короче и без ошибок в расчёте.

Правило: ^N — N-й элемент с конца, счёт с конца начинается с 1.

БЛОК 1 • ЛОВУШКА

Ловушка: ^0

Program.cs

```
string[] songs = { "Believer", "Numb", "Thunder", "Faded" };
Console.WriteLine(songs[^0]); // «нулевая с конца»?
```

Вывод в консоли

```
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in C:\Projects\ArraysDemo\Program.cs:line 2
```

Перевод: необработанное исключение — индекс вышел за границы массива. Ошибка в строке 2.

Почему

^0 — это songs[songs.Length], то есть songs[4]. Такой ячейки нет: она «сразу за последней».

Как правильно

Последний элемент — songs[^1]. Обычный счёт начинается с 0, а счёт с конца — с 1.

Диапазон a..b: кусок массива

Диапазон вырезает из массива подряд идущие элементы — как отрезать несколько вагонов от поезда. Получается новый массив.

Program.cs

```
string[] week = { "Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Вс" };
string[] middle = week[1..4]; // с [1] до [4], [4] не входит
foreach (string day in middle)
{
    Console.WriteLine(day);
}
Console.WriteLine(middle.Length);
```

Вывод в консоли

```
Вт
Ср
Чт
3
```

индекс	week	в [1..4]?
0	Пн	нет
1	Вт	да
2	Ср	да
3	Чт	да
4	Пт	нет, конец

Вывести диапазон целиком через Console.WriteLine(middle) нельзя: напечатается System.String[]. Нужен цикл.

Правило: в a..b начало входит, конец не входит. Элементов получится b – a.

Короткие диапазоны

Начало или конец можно не писать: без начала — «с самого начала», без конца — «до самого конца». В диапазоне работает и ^.

запись	что получится	смысл
week[..5]	Пн Вт Ср Чт Пт	первые 5: будни
week[5..]	Сб Вс	с [5] до конца
week[^3..]	Пт Сб Вс	последние 3
week[..^1]	Пн … Сб	все, кроме последнего
week[..]	все 7 дней	копия массива

Program.cs

```
string[] week = { "Пн", "Вт", "Ср", "Чт",
    "Пт", "Сб", "Вс" };
string[] weekend = week[5..];
string[] last3 = week[^3..];
Console.WriteLine(weekend.Length);
Console.WriteLine(last3[0]);
```

Вывод в консоли

```
2
Пт
```

Правило: [^3..] — последние три, [..3] — первые три, [..] — копия.

Array.Sort: расставить по порядку

Array — набор готовых команд для работы с массивами. Array.Sort расставляет элементы по возрастанию: числа от меньшего к большему, строки по алфавиту.

Program.cs

```
int[] scores = { 340, 120, 560, 210 };
string[] names = { "Яна", "Аня", "Макс", "Лиза" };
Array.Sort(scores); // меняет сам массив
Array.Sort(names);
```

Ошибка: scores = Array.Sort(scores); — CS0029: Cannot implicitly convert type 'void' to 'int[]'. Sort ничего не возвращает, он меняет массив на месте.

i	scores до	после	names до	после
0	340	120	Яна	Аня
1	120	210	Аня	Лиза
2	560	340	Макс	Макс
3	210	560	Лиза	Яна

После сортировки самый маленький — [0], самый большой — [^1]. Исходный порядок пропал.

Правило: Array.Sort(массив) меняет сам массив, по возрастанию.

Array.Reverse и Array.IndexOf

Program.cs

```
string[] names = { "Аня", "Лиза", "Макс", "Яна" };
// где Макс? а Олег?
Console.WriteLine(Array.IndexOf(names, "Макс"));
Console.WriteLine(Array.IndexOf(names, "Олег"));
Array.Reverse(names); // задом наперёд
Console.WriteLine(names[0]);
```

Вывод в консоли

```
2
-1
Яна
```

IndexOf — найти номер

Возвращает индекс первого совпадения. Нет такого элемента — возвращает –1: такого индекса не бывает.

Reverse — перевернуть

Меняет сам массив: первый становится последним. Sort, затем Reverse — порядок по убыванию.

Правило: перед тем как брать элемент по номеру из IndexOf, проверь, что номер не –1.

БЛОК 1 • ЛОВУШКА

Ловушка: имена «отрываются» от очков

Имена и очки лежат в двух массивах: names[i] набрал scores[i]. Сначала у Макса 340, у Лизы 210.

Ошибка: отсортировали scores

Program.cs

```
string[] names = { "Аня", "Макс", "Лиза" };
int[] scores = { 120, 340, 210 };
Array.Sort(scores);
for (int i = 0; i < names.Length; i++)
{
    Console.WriteLine($"{names[i]}: {scores[i]}");
}
```

```
Аня: 120
Макс: 210
Лиза: 340
```

Правильно: сортируем копию

Program.cs

```
string[] names = { "Аня", "Макс", "Лиза" };
int[] scores = { 120, 340, 210 };
int[] sorted = scores[..]; // копия
Array.Sort(sorted);
int best = Array.IndexOf(scores, sorted[^1]);
Console.WriteLine($"Лучший: {names[best]}, {sorted[^1]}");
```

```
Лучший: Макс, 340
```

Правило: связанные массивы не сортируют по отдельности. Сортируй копию scores[..].

Живой кодинг: «Забег на 60 метров»

1. Массив times: время 6 бегунов в секундах.
2. Сортируем: Array.Sort(times).
3. Выводим победителя: times[0].
4. Выводим последнее место через индекс с конца.
5. Здесь будет ошибка. Читаем сообщение и исправляем.
6. Призёры: диапазон times[..3] и цикл с номерами мест.

Место для скриншота: окно исключения IndexOutOfRangeException в Visual Studio

Вывод в консоли (в конце)

```
Победитель: 8,4 с
Последнее место: 10,1 с
Призёры:
1 место: 8,4 с
2 место: 8,7 с
3 место: 8,9 с
```

В коде дробные пишутся через точку: 8.4. На экране — через запятую: 8,4.

Вопросы по блоку 1

Вопрос 1

Что окажется в массиве part?

```
int[] nums = { 10, 20, 30, 40, 50 };
int[] part = nums[1..3];
```

Вопрос 2

Почему nums[^1] надёжнее, чем nums[4]? Что будет с каждым вариантом, если добавить в массив шестое число?

Вопрос 3

Что вернёт Array.IndexOf, если элемента нет? Что случится, если сразу взять элемент по этому номеру?

Вопрос 4

После Array.Sort(scores) имена в names перестали совпадать с очками. Почему? Как этого избежать?

Задание: «Таблица рекордов»

- Программа спрашивает 8 результатов в твоей любимой игре и сохраняет их в массив.
- Выводит все результаты по возрастанию в одну строку, затем лучший и три лучших.
- Спрашивает, какой результат найти, и пишет его место в таблице или «Такого результата нет».

Подсказка

Строку из чисел собирай так: line += score + " "; внутри foreach. Лучший — [^1], три лучших — [^3..]. Место = Length – индекс.

★ **Успел раньше?** Выведи таблицу от лучшего к худшему с местами: «1 место: 2600», «2 место: 2300» и так далее.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Что должно получиться

Вывод в консоли

```
Результат 1:
1200   <- ввёл пользователь
Результат 2:
900   <- ввёл пользователь
Результат 3:
2600   <- ввёл пользователь
Результат 4:
1500   <- ввёл пользователь
Результат 5:
2000   <- ввёл пользователь
Результат 6:
1100   <- ввёл пользователь
Результат 7:
2300   <- ввёл пользователь
Результат 8:
1750   <- ввёл пользователь
По возрастанию: 900 1100 1200 1500 1750 2000 2300 2600
Лучший: 2600
Три лучших: 2000 2300 2600
Какой результат найти?
1500   <- ввёл пользователь
Результат 1500: 5 место
```

Стрелка — пометка для тебя, в программе её нет.

Проверь себя

- Лучший результат (2600) — 1 место, худший (900) — 8 место.
- Если ввести 777, программа пишет «Такого результата нет» и не падает.
- Таблица выводится одной строкой, а не System.Int32[].

★★ «Карусель»

Очередь на аттракцион:

```
string[] queue = { "Аня", "Макс", "Лиза", "Тимур", "Ева" };
```

Каждый круг первый в очереди катается и встаёт в конец. Пользователь вводит, сколько прошло кругов, — программа показывает новую очередь.

Условие: без методов Array, без второго массива — только одна запасная переменная.

Подумай: что выведется после 5 кругов? А после 1000?

Вывод в консоли

```
Сколько прошло кругов?
2   <- ввёл пользователь
Лиза Тимур Ева Аня Макс
```

Стрелка — пометка для тебя, в программе её нет.

БЛОК 2 ИЗ 2 • УРОК 2

List — массив, который растёт

В этом блоке ты научишься:

- создавать список и добавлять в него элементы;
- удалять элементы и узнавать, сколько их сейчас;
- выбирать, что подходит задаче: массив или List.

List: «резиновый массив»

List (список) — ряд элементов одного типа, как массив, только его размер меняется: элементы можно добавлять и удалять. Как список гостей в заметках телефона.

Было: массив на 5 мест

Program.cs

```
string[] guests = new string[5];
// пришёл шестой гость: места нет
```

Стало: список

Program.cs

```
List<string> guests = new List<string>(); // пустой
List<int> scores = new List<int> { 120, 340 }; // сразу с числами
```

Правило: заранее неизвестно, сколько будет элементов, — бери List.

Пока просто запомни

<string> и <int> в угловых скобках говорят, какого типа элементы в списке. Записывай так и не думай почему. Подробно — в теме 5.3.

Размер при создании не указывают: новый список пуст и растёт сам.

Add — добавить, Count — сколько сейчас

Program.cs

```
List<string> guests = new List<string>();
guests.Add("Аня"); // в конец списка
guests.Add("Макс");
guests.Add("Лиза");
Console.WriteLine(guests.Count); // сколько гостей
Console.WriteLine(guests[0]); // индексы как у массива
Console.WriteLine(guests[^1]); // и с конца тоже
```

Вывод в консоли

```
3
Аня
Лиза
```

после	guests	Count
создания	пусто	0
Add("Аня")	[0] Аня	1
Add("Макс")	[0] Аня, [1] Макс	2
Add("Лиза")	Аня, Макс, Лиза	3

У массива — Length, у списка — Count. Перебор: for до i < guests.Count или foreach.

Правило: Add кладёт элемент в конец, Count растёт на 1.

Remove — удалить по значению

Program.cs

```
List<string> guests = new List<string> { "Аня", "Макс", "Лиза" };
bool removed = guests.Remove("Аня"); // Аня передумала
Console.WriteLine(removed);
Console.WriteLine(guests.Remove("Олег")); // Олега нет
Console.WriteLine(guests[0]); // кто теперь первый?
Console.WriteLine(guests.Count);
```

Вывод в консоли

```
True
False
Макс
2
```

До Remove("Аня"):



После: остальные сдвинулись влево



Remove удаляет первое совпадение и отвечает: True — удалил, False — такого не было. Программа не падает.

Правило: после Remove номера элементов за удалённым уменьшаются на 1.

Ловушки списка

1. Номер за пределами Count

Program.cs

```
List<string> guests =
    new List<string> { "Аня", "Макс" };
Console.WriteLine(guests[2]);
```

Вывод в консоли (начало)

```
Unhandled exception. System.ArgumentOutOfRangeException: Index was out of
range. Must be non-negative and less than the size of the collection.
(Parameter 'index')
```

Перевод: индекс вне диапазона, он должен быть от 0 и меньше размера коллекции.

2. Remove внутри foreach

Program.cs

```
List<string> guests =
    new List<string> { "Аня", "Макс" };
foreach (string guest in guests)
{
    if (guest == "Аня")
    {
        guests.Remove(guest);
    }
}
```

Вывод в консоли (начало)

```
Unhandled exception. System.InvalidOperationException: Collection was
modified; enumeration operation may not execute.
```

Перевод: коллекцию изменили, перебор продолжить нельзя.

Как правильно: удалять без перебора — guests.Remove("Аня");. Последний гость — guests[^1] или guests[guests.Count - 1].

Массив или List?

	Массив	List
Размер	задаётся при создании, не меняется	растёт и уменьшается
Создание	new int[5]	new List<int>()
Сколько элементов	Length	Count
Добавить, удалить	нельзя	Add, Remove
[i], [^1], foreach	да	да
[1..3], Array.Sort	да	нет, свои методы — в теме 5.2

Что выбрать?

Дни недели: **массив**, их всегда 7

Корзина в магазине: **List**

Ходы в партии: **List**

Ответы теста из 10 вопросов: **массив**

Правило: размер известен и не меняется — массив; элементы добавляются и удаляются — List.

БЛОК 2 • ВОПРОСЫ

Вопросы по блоку 2

Вопрос 1

Что выведет программа?

```
List<int> nums =
    new List<int> { 5, 7, 5 };
nums.Add(9);
nums.Remove(5);
Console.WriteLine(nums.Count);
Console.WriteLine(nums[0]);
```

Вопрос 2

Почему у списка Count, а у массива Length? Чем они отличаются по смыслу?

Вопрос 3

Сообщения в чате — массив или List? А 64 клетки шахматной доски?

Задание: «Список покупок»

- Программа показывает меню: «1: добавить 2: удалить 3: показать 0: выход» и ждёт выбор.
- 1 — спрашивает товар и добавляет его; 2 — спрашивает товар и удаляет. Если его нет, пишет «Такого нет в списке».
- 3 — показывает список с номерами «1. Молоко». Пустой список — «Список пуст». Меню повторяется, пока не выбран 0.

Подсказка

Меню — цикл do while (тема 1.5). Remove отвечает True или False — по этому ответу понятно, был ли товар.

★ **Успел раньше?** Не добавляй товар, который уже есть в списке: проверь через foreach и напиши «Уже в списке».

Что должно получиться

Вывод в консоли

```
1: добавить 2: удалить 3: показать 0: выход
3   <- ввёл пользователь
Список пуст
1: добавить 2: удалить 3: показать 0: выход
1   <- ввёл пользователь
Что добавить?
Молоко   <- ввёл пользователь
1: добавить 2: удалить 3: показать 0: выход
1   <- ввёл пользователь
Что добавить?
Хлеб   <- ввёл пользователь
1: добавить 2: удалить 3: показать 0: выход
2   <- ввёл пользователь
Что удалить?
Сыр   <- ввёл пользователь
Такого нет в списке
1: добавить 2: удалить 3: показать 0: выход
3   <- ввёл пользователь
1. Молоко
2. Хлеб
1: добавить 2: удалить 3: показать 0: выход
0   <- ввёл пользователь
```

Стрелка — пометка для тебя, в программе её нет.

Проверь себя

- Пустой список: «Список пуст», а не пустота на экране.
- Удаление несуществующего товара не ломает программу.
- После удаления номера в списке идут подряд: 1, 2, 3.

★★ «Чистка плейлиста»

В плейлисте одна песня надоела и встречается несколько раз:

```
List<string> playlist = new List<string>
    { "Numb", "Faded", "Numb", "Believer", "Numb" };
```

Пользователь вводит название. Программа удаляет все копии этой песни, сообщает, сколько удалено, и показывает, что осталось.

Условие: программа не падает с InvalidOperationException. Подумай, что тебе подсказывает ответ Remove.

Вывод в консоли

```
Какую песню убрать?
Numb    <- ввёл пользователь
Удалено: 3
Осталось:
Faded
Believer
```

Стрелка — пометка для тебя, в программе её нет.

ФИНАЛ • ВОПРОСЫ ПО ВСЕЙ ТЕМЕ 2.1

Вопросы по теме

Вопрос 1

В коде заменили marks[marks.Length - 1] на marks[^0]. Что изменится, когда программа дойдёт до этой строки?

Вопрос 2

Поле «Морского боя» 10 на 10 и таблица рекордов, куда добавляются новые игроки: где массив, а где List? Почему?

Вопрос 3

Ты нашёл bestIndex лучшего игрока циклом из занятия 1, а потом вызвал Array.Sort(scores). Можно ли ещё верить names[bestIndex]?

Мини-проект: «Бинго»

Ты заполняешь карточку: 5 чисел от 1 до 30. В мешке лежат шары с числами от 1 до 30.

Компьютер достаёт 10 шаров. Вынутый шар в мешок не возвращается, поэтому повторов нет.

Программа отмечает шары, которые есть в твоей карточке, и в конце показывает счёт.

Это пригодится в Проекте 2 «Игра на поле»: таблица рекордов там хранится в List.

Вывод в консоли

```
Число 1 для карточки (от 1 до 30):
17   <- ввёл пользователь
Число 2 для карточки (от 1 до 30):
3   <- ввёл пользователь
Число 3 для карточки (от 1 до 30):
25   <- ввёл пользователь
Число 4 для карточки (от 1 до 30):
12   <- ввёл пользователь
Число 5 для карточки (от 1 до 30):
5   <- ввёл пользователь
Твоя карточка: 3 5 12 17 25
Шар 1: 17, есть в карточке!
Шар 2: 8
Шар 3: 29
Шар 4: 3, есть в карточке!
Шар 5: 14
Шар 6: 22
Шар 7: 1
Шар 8: 30
Шар 9: 11
Шар 10: 25, есть в карточке!
Совпадений: 3 из 5
Последний шар: 25
Осталось в мешке: 20
```

Стрелка — пометка для тебя, в программе её нет. Шары случайные: у тебя будут другие.

Что должна уметь программа

1

Спрашивает 5 чисел для карточки и переспрашивает, если ввели буквы или число не от 1 до 30.

2

Показывает карточку по возрастанию в одну строку.

3

Достаёт 10 разных шаров и у каждого пишет, есть ли он в карточке.

4

В конце пишет число совпадений, последний шар и сколько шаров осталось в мешке.

План по шагам

Шаг	Что сделать	Тема	Как проверить
1	Ввод карточки уже в заготовке. Отсортируй card и выведи её в одну строку	1.2, 2.1	Введи 17, 3, 25, 12, 5 — видишь «Твоя карточка: 3 5 12 17 25»
2	Создай мешок List<int> balls и в цикле положи в него числа от 1 до 30	1.5, 2.1	Временно выведи balls.Count — должно быть 30
3	10 раз: случайный шар из мешка, убери его из balls, добавь в drawn, проверь через IndexOf	1.4, 1.5, 2.1	Шары не повторяются; у числа из карточки есть «есть в карточке!»
4	Выведи совпадения, последний шар drawn[^1] и balls.Count	2.1	Осталось в мешке ровно 20

Заготовка

Program.cs

```
Random random = new Random();
int[] card = new int[5];
for (int i = 0; i < card.Length; i++)
{
    int number;
    do
    {
        Console.WriteLine($"Число {i + 1} для карточки (от 1 до 30):");
    } while (!int.TryParse(Console.ReadLine(), out number) || number < 1 || number > 30);
    card[i] = number;
}
// Шаг 1. Отсортируй card и собери строку «Твоя карточка: …»
// Шаг 2. Мешок: List<int> balls с числами от 1 до 30, пустой List<int> drawn
// Шаг 3. 10 раз: шар balls[random.Next(0, balls.Count)], Remove, Add, IndexOf
```

Что делает do while

Спрашивает число снова, пока ввели не число или число не от 1 до 30.

Шаг 4

Совпадения считай счётчиком hits++ (занятие 1), последний шар — drawn[^1].

Готово, если…

- Если ввести «abc» или 40, программа снова спрашивает то же число.
- Карточка 17, 3, 25, 12, 5 выводится как «3 5 12 17 25».
- Среди 10 шаров нет одинаковых, в конце «Осталось в мешке: 20».
- Если шара нет в карточке, программа не падает и не пишет «есть».

★ Без повторов в карточке

Ввели число, которое уже есть в карточке, — «Это число уже есть», спросить снова.

★★ До победы

Шары достаются, пока не совпадут все 5 чисел: «Бинго! Понадобилось 23 шара».

Каждую строку своего кода ты должен уметь объяснить. Готово — сохрани в Git и отправь на GitHub.

Челлендж: «Музыкальный автомат»

Программа хранит плейлист в List и понимает команды:

- add — добавить песню в конец, del — удалить песню;
- next — сыграть первую песню: она уходит в конец очереди и попадает в «историю»;
- history — показать 3 последние сыгранные песни;
- shuffle — перемешать плейлист без готовых методов;
- show — показать плейлист с номерами, exit — выход.

Подумай, что должна делать команда next, если плейлист пуст.

ИТОГИ ТЕМЫ 2.1

Теперь я умею

- Хранить много значений в массиве, брать их по индексу и не выходить за границу.
- Перебирать данные циклами for и foreach и выбирать нужный цикл.
- Считать сумму и среднее, находить лучший и худший результат и его номер.
- Брать элементы с конца и куски массива, сортировать, переворачивать и искать.
- Хранить данные, которые добавляются и удаляются, в списке List.
- Решать, что подходит задаче: массив или List.